Group 55 Bank Customer Churn Project

Purpose and current status

The project estimates the probability that a customer exits a bank from ten demographic and account attributes. It supports a study of risk ranking and classification thresholds; it does not establish that a retention offer causes customers to stay. The current deliverable is a progress milestone, not a completed deployment or an independent final evaluation.

The repository's original experiment is the authoritative ANN reference: a Keras network with 12 encoded inputs, two six-unit ReLU hidden layers and one sigmoid output, trained for 100 epochs with Adam and batch size 32. Its 127 parameters and seed-0 split differ from the earlier assistant-generated scikit-learn experiment. The two experiments must not be merged into one results table as if they used an identical protocol.

Data and representation

The CSV contains 10,000 labeled customers, including 2,037 churners. The three identifiers are excluded from model input. Geography is one-hot encoded; gender is ordinal encoded; the other predictors remain numeric. A standard scaler is fitted on the training records after encoding. It also scales indicator columns. The original representation is retained so existing results remain interpretable.

Loading validates required fields, complete inputs, both target classes, finite numeric values, binary account flags and nonnegative account quantities. Repeated customer IDs are rejected to prevent customer leakage across partitions. All rows in the supplied dataset pass. These checks do not authenticate the data source or resolve its missing prediction horizon.

Implementation

- churn_data.py: loading, schema checks, stratified splitting and preprocessing.

- churn_models.py: Keras ANN adapter and model registry, with majority-prior, random forest, logistic regression and XGBoost factories.

- churn_evaluation.py: reusable fitted pipeline and metric calculations.

- churn_figures.py: class distribution, training-loss, precision-recall and confusion-matrix figures.

- artificial_neural_network.py: command-line orchestration and evidence export.

- build_comparison_table.py: a LaTeX table generated from the saved extension metrics.

- test_churn.py: behavioral tests for split safety, input validation, threshold semantics and frozen preprocessing.

The majority baseline predicts the training churn prevalence, so its classification is always retention at the shared strict decision rule $p > 0.5$. This gives a meaningful constant-probability benchmark as well as majority-class accuracy. Logistic regression tests a linear boundary. The random forest retains 200 trees, now fitted with two CPU workers. XGBoost uses 300 trees, depth 3, learning rate 0.05, row/column subsampling 0.8 and L2 coefficient 1. The ANN architecture is preserved rather than silently replaced.

Evidence and evaluation

results/metrics.json and its figures preserve the original experiment. That run used a stratified 7000/1500/1500 split and evaluated both validation and test. Its ANN validation AP was 0.694 and test AP was 0.676; the forest validation AP was 0.664 and test AP was 0.672. These are historical repository results, not newly generated claims.

The new results-extension/ directory records a separate validation-only comparison. Its metrics, package versions, model parameters, dataset checksum, row assignments and raw probabilities make the experiment auditable. The progress report's new comparison table is generated from that record. No statistical superiority is claimed from one split.

Routine runs now default to validation only. An explicit --evaluate-test option is required to compute test predictions. This prevents accidental repeated test use but cannot undo the fact that the original test scores were inspected. Future tuning should use development folds; an external cohort would provide stronger independent evidence. If no independent cohort is available, the final report must retain this limitation.

Accuracy, precision, recall, F1, average precision, ROC AUC, Brier score and log loss serve distinct purposes. Threshold-dependent metrics use the same strict cutoff. A fixed 0.30 validation probe quantifies additional true and false positives without presenting it as an optimized policy. Predictions must be finite probabilities between zero and one. Invalid probabilities are rejected rather than silently clipped into plausible-looking metrics.

Reproducibility and safeguards

Each new run writes to a separate directory and refuses to overwrite an existing metrics file. Saved pipelines include preprocessing, enabling inference from raw named columns. Run tests with python -m unittest discover -s code -p 'test_*.py'. The GitHub workflow runs those tests without downloading TensorFlow because the ANN import is lazy.

The original ANN had no early stopping or validation-loss monitoring. Training loss alone does not rule out overfitting. The revised code preserves that baseline; testing early stopping, dropout, class weighting and calibration remains future work. Library and hardware differences may change neural results even with deterministic seeds. Saved pickle-based models should be loaded only from trusted sources with matching dependencies.

Remaining work and submission

Next steps are bounded cross-validation searches, repeated seeds, operating thresholds at an explicit precision or contact-budget target, calibration checks and subgroup error analysis with sample counts. These are proposed experiments, not completed outcomes. No causal or deployment-readiness claim follows from the current benchmark.

For the current course milestone, submit the compiled ACL progress report, code and dataset in one ZIP. The dataset is below 50 MB. The source report, bibliography, figures and setup instructions are included for review and rebuilding. Both members should verify the existing contribution statements before submission. This revision was prepared with AI assistance.

ANN runtime verification

The repository ANN was rerun for all 100 epochs with the revised validation-only runner. It completed with validation AP 0.694 and F1 0.560, close to the historical AP 0.694 and F1 0.561. The new metrics, probabilities and environment metadata are stored separately in results-ann-check/. Historical figures and values were not overwritten. This verifies execution of the actual Keras path; it does not establish platform-independent bitwise reproducibility or a new independent test score.

Recorded extension results

Model	F1	AP	ROC AUC	Brier
majority	0.000	0.204	0.500	0.162
logistic	0.302	0.480	0.774	0.135
random-forest	0.565	0.665	0.843	0.108
xgboost	0.575	0.694	0.863	0.103